Code Design and Performance

CardGame

The CardGame class is the main game class, containing the main function used for running the code. It has 2 private static attributes which are ArrayLists storing the players and decks inside the current game. These attributes have public getter methods, and players has a setter method used exclusively for testing purposes.

The main method is used to start the game. It takes 2 inputs, the number of players which is checked to be a non-negative integer before preceding, and a file path for the pack file to be used. The loadPack function is used to read this file, returning an empty ArrayList if the pack doesn't exist or is invalid, such as containing integers <= 0 or being the wrong size for the number of players chosen. Once both inputs have been taken and verified, the main function will call private methods inside CardGame, starting with createDecks and addPlayers, which fill the private players and decks attributes, assigning each player their left and right deck on creation. 4 cards are then dealt to each player using the dealCardsToPlayers function, and the dealCardsToDeck does the same with the remaining cards in the pack. Once everything is set, each player thread is started.

Once the game is won, the static method gameOver will be called by the winning thread, passing through its identifier to indicate it is the winner. This can then be displayed in the console. The main method can then interrupt all other player threads to make sure they finish up. This prevents any issues where players are waiting for decks to have cards available to take, as it will wake them up and cause them to start checking if the game is over again, at which point they should clean themselves up and exit. The method then calls each deck's outputDeck function to create the output files for the deck.

Card

Card is a simple class representing a card in the game and holding a private integer value, the face value of the card, which is set at object creation. This attribute has a getter method getValue(), and the toString() method returns the value of the card as a string to be used in the toString() method of the CardCollection class.

CardCollection

Card Collection is created as an abstract model to be used by both the Deck and Hand since they rely on similar underlying methods. It implements a LinkedBlockingQueue, which is part of the java.util.concurrent library. This functions similarly to a normal queue, however is built to be thread safe, with waiting built-in so threads wait until the queue contains a value before taking anything out. These methods can also be interrupted like the standard wait() method for synchronisation. This blocking queue is protected so that it can be used inside concrete methods for the Deck and Hand classes.

It implements fundamental methods to get, add and remove cards from the queue. The size of the queue can also be accessed, along with getting an iterator object that can be used to iterate through the queue inside functions outside Hand and Deck, for example, in the checkWin() method of the Player class. The toString method is also overwritten for this class, outputting the value of the cards in the queue, separated by a space. This toString method is used by both the decks and hands.

Deck

Deck extends the abstract CardCollection method, adding a private identifier to represent which deck the object refers to, e.g: Deck 1. This identifier is generated automatically at object instantiation using a static AtomicInteger counter. The only additional methods added by deck are a getter method for the identifier (getIdentifier()), and an outputDeck() function which uses a buffered FileWriter to output the contents of the deck, using the classes toString() method to simplify the process. This method is designed to attempt to write to the file 3 times should an IOException occur, ensuring the output is consistent at the end of each game.

Hand

Hand is a simpler extension of CardCollection compared to deck. It has a second removeCard function allowing a Card object to be removed from anywhere in the queue, provided that queue contains the Card. It also implements a getRandomCard() function that returns a random card from inside the queue which can be used when deciding what card to discard during the round. This uses the built-in Java.util.Random module for generating a random integer.

Player

The Player class represents the threads within the application. It extends the Thread class, and overrides the built-in run() method with the game play, such that it initially checks the hand it's been dealt to see if it wins, before continually taking a card from the deck to it's "left", i.e. the deck with the same identifier as it, and outputting to the deck to the right (deck with an identifier 1 greater than the players identifier or 1 if it has the largest player identifier in the game), completing the circular setup required for the card game. Each gameplay turn is done as an atomic action, only starting if the game is not over, and checking if the player has won at the end of each turn. This ensures all players always finish the game with 4 cards in their hand. drawCard() can be used to place a card into a players hand during the start of the game.

Similarly to Deck, the Player class implements an auto generated unique identifier using a static AtomicInteger counter so that it's thread safe. Each player has a unique instance of the Hand class, stored a private attribute with a getter method getHand() used for testing only. The "left" and "right" decks are passed in when a new player is instantiated and stored as private attributes, similarly with getter methods for testing purposes. There is also a public getter for the identifier attribute. Other methods, such as startOutput() and writeOutput() are used for handling the output file stream, which is stored as a private attribute so that it can be accessed by any method.

To track the winner of the game, the Player class implements a second static AtomicInteger called winner. This is initialised as -1 to indicate no winner and set to the identifier of the winning player once a player thread decides it has a winning hand. The static method checkGameOver() returns a boolean output depending on if the winner is still -1 or not. When a player wins, it calls the gameOver() method of the CardGame class, which handles interrupting all other threads, allowing them to finish off their turn. As they check if the game is over, they should finish up after that turn. One performance issue found is that players will be able to undertake multiple turns until the change in winner is detected due to different threads running at different speeds. Players can also get stuck if it's thread speeds off and empties the deck, causing it to have to wait for another card to be put in by the player to it's left, although it can be argued that this keeps the game fairer between players.